Architecture Overview — Data Science Projects with Visualization Dashboards

> Training document 0. This is the map; each area below is its own independent curriculum — a folder of focused documents you can work through on its own:
>
> | Area | Folder | Documents |
> |---|---|---|
> | Data Science / ML | `01-data-science-ml/` | EDA & data engineering → feature engineering → regression models → time-series forecasting → multi-source pipelines |
> | Python API Backend | `02-python-api-backend/` | HTTP/REST & FastAPI fundamentals → API design & best practices → testing → deployment |
> | Streamlit Dashboards | `03-streamlit-frontend/` | Core concepts → charting & visualization → testing → deployment |
> | React Front Ends | `04-react-frontend/` | Core concepts → routing & API integration → testing → deployment → patterns for data-heavy dashboards |
> | MCP Servers | `05-mcp-server/` | Core concepts → tool design → testing → deployment |
> | Conversational Agents | `06-conversational-agent/` | Core concepts → tool calling, guardrails & generative UI → testing → deployment |

This document explains the general pattern that ties all six areas together, the architectural principles behind it, and the design options available at each layer — read it first, then pick whichever area's folder you need.

1. The general shape of this kind of project

A common pattern for turning a data analysis into something other people can actually use looks like this:

1. **Acquire and understand a dataset.** Load it, profile it (missing values, coverage, distributions), and decide what subset of it is actually relevant to the question being asked. Mature projects add *further* external sources for context (see `01-data-science-ml/05-multi-source-pipelines.md`), each validated before publishing and recorded with its provenance and licence.
2. **Engineer features and train models.** Transform raw data into inputs a model can use, then fit and compare models against a held-out test set.
3. **Persist the results.** Save the cleaned data, the engineered features, and the model outputs (predictions, forecasts, comparison metrics) somewhere durable, rather than keeping everything only in memory. Where the data must stay current, this step is also **scheduled** (a refresh job that validates, re-runs, restarts what caches, and alerts — §15).
4. **Serve the results.** Expose those persisted results through an interface other software can call — typically a web API that returns structured data (usually JSON) over HTTP.
5. **Present the results.** Build a dashboard or application that calls that interface (or, in a simpler setup, reads the persisted results directly) and renders it as charts, tables, and interactive controls for a human to

explore.

6. **Optionally, expose the results to an LLM agent.** Wrap step 4's API as a set of tools an LLM can call (see `05-mcp-server/`), and build a conversational agent on top of that tool layer (see `06-conversational-agent/`) so the same underlying results can also be explored through open-ended natural-language questions, not only through the fixed charts/controls a dashboard offers. This is a genuinely optional sixth step, layered on top of step 4 rather than replacing step 5 — most projects of this shape stop at step 5, and adding step 6 only pays for itself once open-ended, unpredictable questions are common enough that a fixed dashboard's controls start to feel limiting.

Steps 1–2 are **data science** work; step 3 is where data engineering discipline matters (a reproducible, well-defined hand-off, not ad-hoc in-memory state); steps 4–5 are **software engineering** — building durable, testable interfaces and user-facing applications around what the data science produced; step 6, where present, is an additional software-engineering layer built *on top of* step 4's contract, not a replacement for any earlier step.

2. Architecture principles and best practices

These principles recur across every layer discussed in this series, regardless of which specific technology implements a given layer. Understanding them generally will make each area's own document easier to place in context.

2.1 Separation of concerns

Each layer should have one clear responsibility and should not need to understand *how* the layers around it are implemented — only the **contract** it exchanges with them. The data science layer's job ends at "produce a well-defined output"; it doesn't need to know whether that output ends up in a Streamlit app or a React dashboard. The API layer's job ends at "expose that output as JSON over HTTP"; it doesn't need to know how the data was modeled. This is what lets each layer be built, tested, and changed by different people, at different times, without everyone needing to hold the whole system in their head at once.

2.2 Loose coupling via explicit contracts

Layers should communicate through **explicit, stable contracts** — a defined file format and column schema between the data layer and the API, a defined JSON response schema between the API and a frontend — rather than one layer reaching into another's internals. A contract can be versioned, documented, and validated independently of either side's implementation. The cost of this discipline is that changing a contract requires updating everyone who depends on it; the benefit is that anyone who honors the contract can be replaced or extended without the rest of the system knowing or caring.

Contracts still need to evolve, though, and there are established strategies for doing that without breaking every consumer at once: prefer **additive, backward-compatible changes** (adding a new optional field rather than renaming or removing an existing one) wherever possible; when a genuinely breaking change is unavoidable, introduce it as an explicitly **versioned** contract (a new API version, a new schema version) served *alongside* the old one for a deprecation period, so consumers can migrate on their own schedule rather than being forced to update in lockstep with the producer. See the API backend curriculum's versioning section for the concrete mechanisms (URL-path vs. header-based versioning) this applies to for an HTTP API specifically.

2.3 Single source of truth

Any value that matters in more than one place (a set of valid categories, a business rule like a cutoff date, a set of column names) should be defined in exactly one place and referenced everywhere else — never redefined inline in multiple spots. When a system spans multiple languages or runtimes (a Python data layer, a Python API, a TypeScript frontend), a literal shared import usually isn't possible across all of them, so the "single source of truth" becomes a value that's **defined once and then intentionally, consistently hand-mirrored** wherever the runtime boundary requires it — treated as a real, tracked maintenance surface rather than an assumption that everyone will remember to update all the copies.

2.4 Fail gracefully, fail loudly enough to notice

A system with multiple independently-deployed layers will, at some point, have one layer ready before another (the API is up but the data pipeline hasn't produced its output yet; the frontend is deployed but the API isn't reachable). Two failure modes are both wrong: silently showing stale or wrong data, and crashing uninformatively. The right behavior is to detect the specific failure condition and communicate it clearly — a specific HTTP status code, a specific in-page message — so whoever's looking at it (a developer or an end user) knows exactly what's missing and why, rather than guessing.

This extends to *derived analysis*: when a computation cannot be trusted (a source is stale, an input is missing, a number is not finite), the producer writes the same schema with **explicit nulls and a reason**, and the API answers `503` naming the cause — never a plausible-looking `200`. An old result sitting under a new timestamp is worse than no result. See API Design, §2.3.

2.5 Statelessness where possible

A service that doesn't need to remember anything between requests (no server-side session state) is far easier to scale, restart, and reason about — any request can be handled by any instance of the service, and restarting it loses nothing. The read-only, data-serving API style covered in this series is naturally stateless: every request is a self-contained question answered entirely from persisted data. Keep it that way unless there's a specific, justified reason not to (e.g., real user authentication and sessions, which is out of scope for a pure analytics-serving API). Operational state that genuinely must persist — an admin's choice of which model is live, for instance — belongs in a small, atomically-written store on the one service that owns that setting, not in a shared database added for its sake (§16).

2.6 Idempotency

An operation is **idempotent** if performing it multiple times has the same *effect on server state* as performing it once — this is about side effects, not about whether the response body looks identical every time. `GET` requests (read-only) are idempotent by nature: they have no side effects at all, so repeating one leaves the server in exactly the same state as making it once (trivially — zero changes, every time). That's still true even if the *data* a `GET` returns changes between calls (because something else wrote to it in the meantime) — that's a change caused by whatever did the writing, not by the `GET` itself, so it doesn't break the guarantee. This matters because clients, proxies, and browsers are allowed to retry idempotent requests automatically (e.g., after a network blip) without

asking permission; an operation that *isn't* idempotent (like "increment a counter") must never be exposed as a `GET`, or an automatic retry could silently corrupt data.

`GET`'s idempotency is almost trivial, though, since it has no side effects to begin with — `PUT` is a more instructive example precisely because it *does* write to server state and is still idempotent: `PUT`'s semantics are "replace this resource with exactly this representation," so calling it once or five times with the same body leaves the resource in the identical final state either way — nothing accumulates. Contrast this with `POST`, whose semantics are typically "create a new resource" — submitting the same body to a `POST` endpoint five times usually creates five separate resources, which is exactly the non-idempotent case `PUT` is not. (This guarantee still depends on the server actually implementing "replace" semantics correctly for its `PUT` handlers — the method name alone doesn't enforce it.)

2.7 Configuration over hardcoding

Anything that differs between environments (a database path, an API base URL, a deploy prefix, a secret credential) belongs in **configuration** (environment variables, a config file read at startup) — never hardcoded into source code. This is what lets the *same* built artifact run correctly in a local development environment, a staging environment, and production, without a code change between them (a principle sometimes called "build once, configure per environment," part of the widely-referenced 12-factor app methodology).

2.8 Bounded autonomy

Once a layer's behavior is driven by an LLM deciding what to do next (an MCP server's caller, a conversational agent's tool-calling loop), the earlier principles above still apply, but a new one joins them: **give autonomous, LLM-driven decision-making explicit limits, not just correct logic.** A loop bounded only by "the model decides when to stop" has no upper bound at all if the model's judgment is ever wrong — a hard cap on iterations, a fixed classification step before autonomy is granted, a bounded/annotated response instead of an unbounded one, are all the same underlying move: treat the model's autonomy as something to design a perimeter around, not something to trust unconditionally just because it's usually right. See `06-conversational-agent/02-tool-calling-and-guardrails.md` for what this looks like concretely.

2.9 Provenance, licence and caveats travel with the numbers

Data that came from somebody else arrives with obligations: a licence (sometimes restricting commercial use or derivatives), a required citation, a note that it is a preliminary release. Record those as **data** next to each series when it is ingested, carry them in every API response (a single envelope of `note`, `caveats`, `attribution`, `source_vintage`), and let every later consumer — a second frontend, an agent, an export — pass them through unchanged. Retyping them per consumer guarantees that one of them eventually won't. Licence review is part of choosing a source: a source that embeds a no-derivatives component cannot feed a pipeline that transforms and republishes it. See Multi-Source Pipelines, §4.

2.10 Deterministic code for what must not fail

A refinement of §2.8: wherever an output **must** contain something — a disclosure that a series is preliminary, the correct unit conversion, a rank computed over the right population, a link to the matching page — generate it in

ordinary code from the data, not by asking a model (or a developer, per consumer) to remember it. The LLM then phrases; the code decides the facts and the obligations. See Agent Guardrails, §7–§10.

2.11 Presentation decisions are shared as tokens, not copied as styles

When more than one application (or more than one page) needs to look and behave consistently, put the decisions — colours, spacing, type, component states, accessibility rules — in a **design system**: named *tokens* consumed by a component library, with themes as small token overrides applied at one point (the app root). An application then never hard-codes a colour; it uses semantic tokens, so a theme switch, a contrast fix or a rebrand is a change in one place. The same principles as the rest of this series apply: a single source of truth (§2.3), an explicit contract (the token and component API, §2.2), configuration rather than hard-coding (§2.7) — plus two specific to presentation: make required branding impossible to omit (required props, not defaults), and fix gaps in the shared system rather than patching each app. In this series the front end consumes such a system (the Syena Design System, documented in its own `DESIGN.md`); see React Core Concepts, §7 for how it is structured and consumed, and Patterns for Data-Heavy Dashboards for theming in practice.

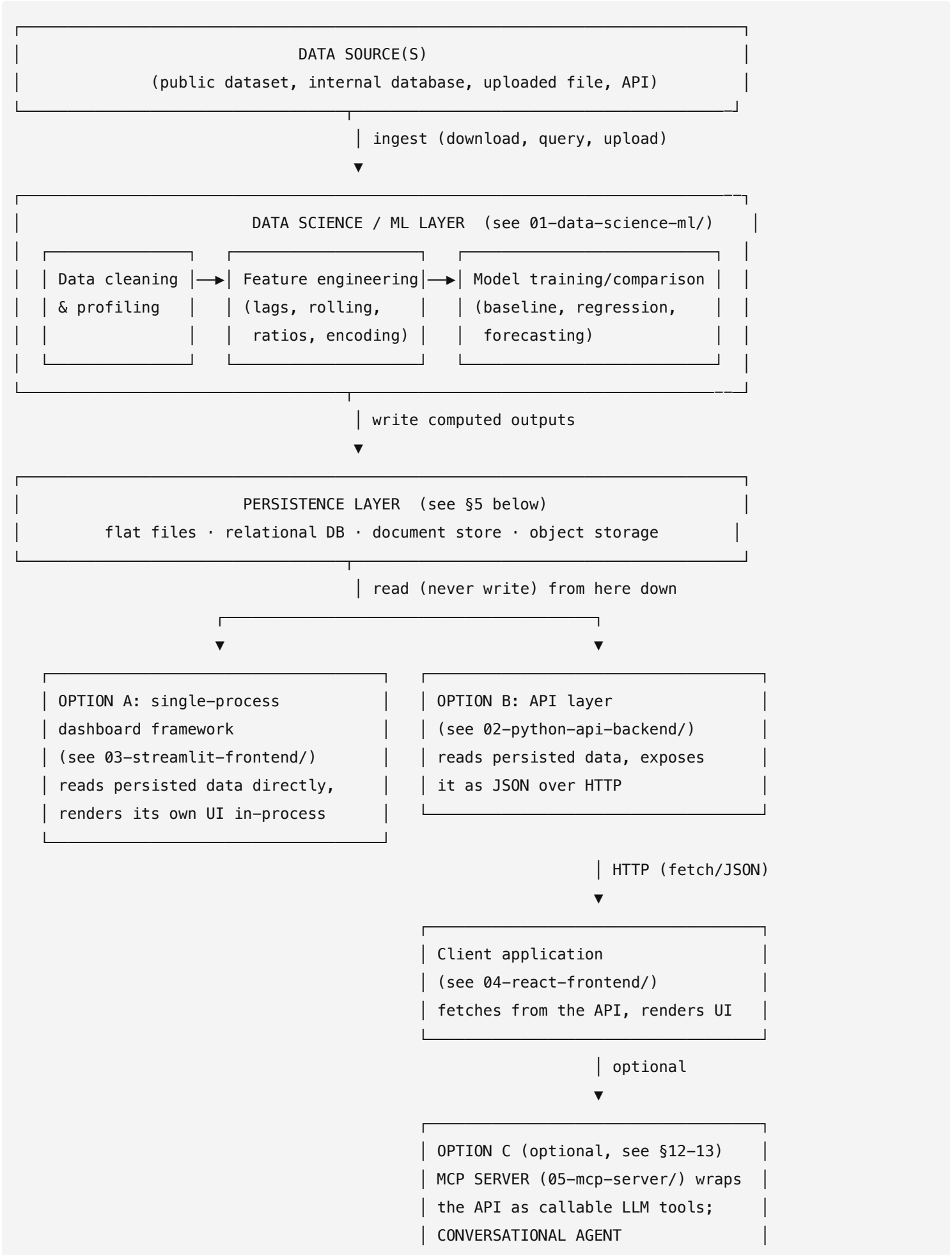
3. Common architecture patterns

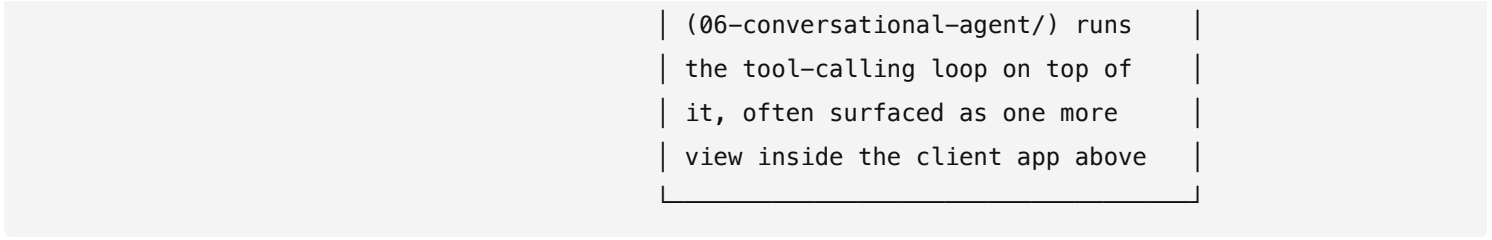
Pattern	What it is	When it fits
Layered architecture	The system is organized into horizontal layers (data → API → presentation), each only depending on the layer directly below it	The default pattern for this whole series — a clear, easy-to-reason-about default for small-to-medium systems
Pipeline / ETL (Extract-Transform-Load)	Data moves through a sequence of discrete, ordered processing stages, each reading the previous stage's output and writing its own	Any batch data-processing workflow — exactly the shape of the data science layer in this series
Medallion architecture (Bronze/Silver/Gold)	A specific, widely-used convention for *naming* the stages of an ETL pipeline by what state the data is in: **Bronze** — raw, unmodified, ingested as-is (kept for lineage/reprocessing); **Silver** — cleaned, validated, deduplicated, filtered, but not yet shaped for any one specific downstream use; **Gold** — curated, aggregated, business/analysis-ready data, consumed directly by dashboards, reports, or models	Any pipeline where it's useful to give each stage's output a shared vocabulary — see `01-data-science-ml/01-eda-data-engineering.md` for how this project's own raw→filtered→feature-engineered flow already *is* this pattern
Client-server	A client (frontend) and a server (API) communicate over a network, with the server owning data/logic and the client owning presentation	Whenever a UI and its data/logic need to be deployed, scaled, or developed independently
Monolith vs. microservices	A monolith is one deployable unit containing all logic; microservices split logic into several independently-deployable services communicating over a network	Monoliths are simpler to build, test, and deploy for small-to-medium systems (start here); microservices add real operational complexity and are justified mainly by needing independent scaling/deployment of specific parts, or by very large, multi-team codebases — not a default to reach for early

Pattern	What it is	When it fits
Request-response vs. event-driven	Request-response: a client asks a specific question and waits for an answer (what a typical REST API does). Event-driven: components react asynchronously to events/messages without a direct request/response pairing	This series is entirely request-response, which is the right default for "answer a question about already-computed data"; event-driven architectures matter more for real-time, high-throughput, or loosely-coupled multi-service systems
Batch vs. streaming	Batch: data is processed in discrete chunks on a schedule or on demand. Streaming: data is processed continuously, record by record, as it arrives	This series is entirely batch (a pipeline runs, produces a snapshot of results, the API serves that snapshot) — appropriate whenever near-real-time freshness isn't a hard requirement
Edge-enforced access policy	Access control, rate limiting and security headers are applied by a gateway/edge layer in front of the services (service tokens for machines, identity-provider login for humans) rather than coded into each service	When several services share one public hostname and the policies differ per path — keeps services simple and policy centrally revocable; see §16
Scheduled refresh with alerting	A job that periodically re-downloads sources, validates, re-runs the pipeline stages as separate failure domains, restarts caches and sends one prioritised notification	Any product whose data goes stale on a known cadence; see §15
Tool-use / agentic loop	An LLM is given a set of callable tools and decides, per turn, whether/which to call and when it has enough to answer, rather than following a fixed call sequence	The shape of the optional sixth layer in this series — see `05-mcp-server/` for exposing tools to a model and `06-conversational-agent/` for the loop that calls them

The overall shape used across this training series — a batch ETL pipeline (Data Science folder), producing files consumed by a layered client-server split (API + frontend folders) — is a **modular monolith with a decoupled presentation layer**: each concern is cleanly separated in code and can be developed independently, without the operational overhead of running many independently-deployed microservices. This is the right default for most small-to-medium data projects; reach for true microservices only once you have a specific, concrete reason (independent scaling needs, separate team ownership, genuinely different technology requirements per service) that a modular monolith can't satisfy.

4. High-level end-to-end architecture diagram


DATA SOURCE(S)
(public dataset, internal database, uploaded file, API)
ingest (download, query, upload)
DATA SCIENCE / ML LAYER (see 01-data-science-ml/)
Data cleaning & profiling
Feature engineering (lags, rolling, ratios, encoding)
Model training/comparison (baseline, regression, forecasting)
write computed outputs
PERSISTENCE LAYER (see §5 below)
flat files · relational DB · document store · object storage
read (never write) from here down
OPTION A: single-process dashboard framework (see 03-streamlit-frontend/) reads persisted data directly, renders its own UI in-process
OPTION B: API layer (see 02-python-api-backend/) reads persisted data, exposes it as JSON over HTTP
HTTP (fetch/JSON)
Client application (see 04-react-frontend/) fetches from the API, renders UI
optional
OPTION C (optional, see §12-13) MCP SERVER (05-mcp-server/) wraps the API as callable LLM tools; CONVERSATIONAL AGENT

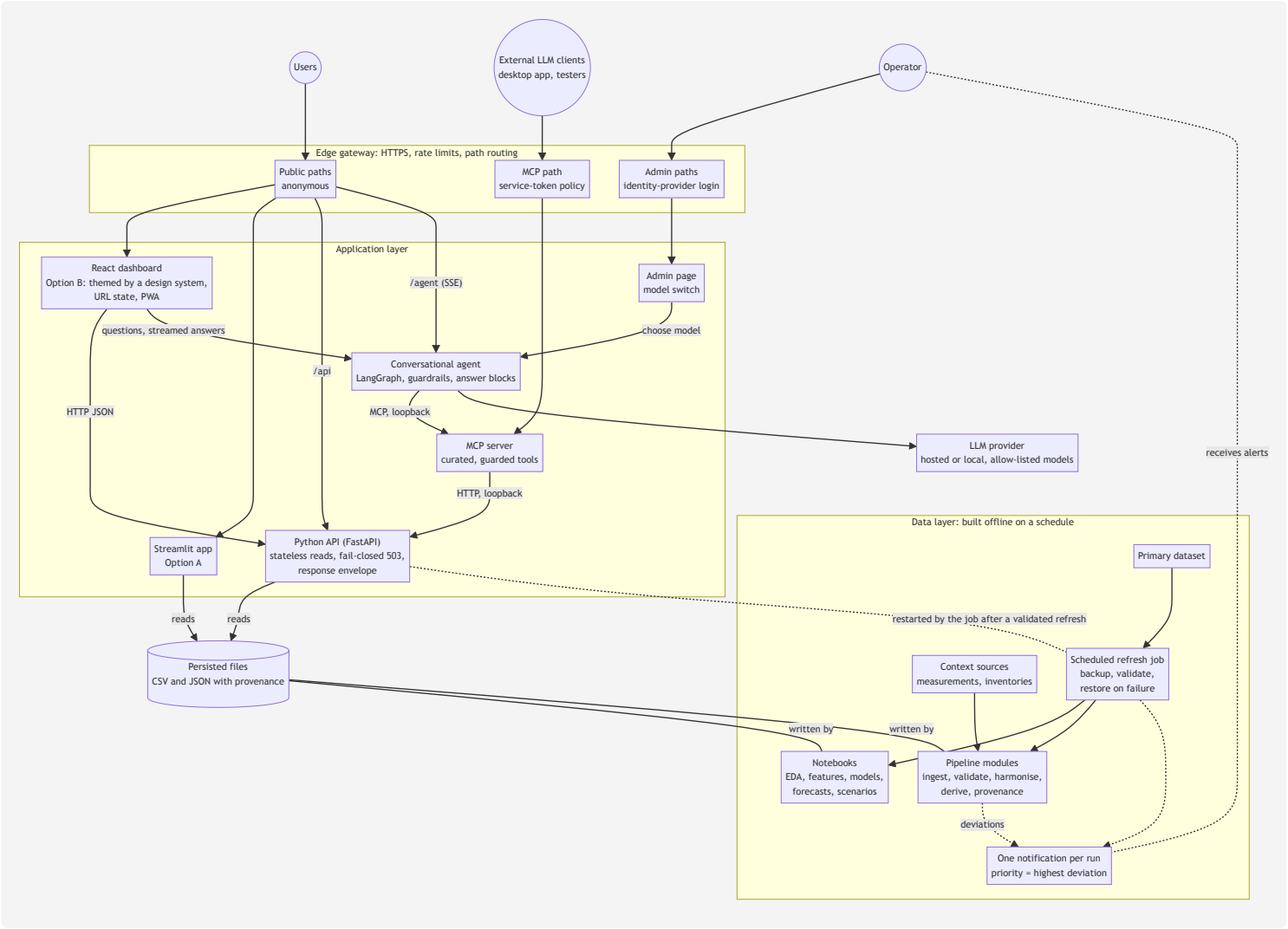


Not drawn above: an **operational layer** around the whole stack — the scheduled refresh that keeps the data layer current and restarts the API afterwards (§15), and an **edge** in front of the public services that applies rate limits and access policies per path (§16). §4.1 draws the complete system with both included.

Option A and Option B are **alternative ways to build the last mile**, not sequential stages — a real project might build only one, or both side by side (see §6). Option C is a further optional addition **on top of** Option B — an MCP server has no persisted-data access of its own; it calls the same API Option B already exposes, so Option C only makes sense once Option B already exists (see §12–13).

4.1 The complete system, with operations and the edge

The diagram below shows every component this series covers and how they connect, including the parts the sketch above leaves out: the scheduled refresh, the alert path, the edge gateway with its different access policies, the admin page, and the LLM provider. Solid arrows are runtime requests or data flow; dotted arrows are operational signals (alerts, restarts, configuration).



How to read it:

Part	What to notice
Data flows one way	Sources → refresh job → notebooks and pipeline → persisted files → API or Streamlit. Nothing downstream writes back to the data layer (§9).
The API is the hub	The React dashboard and the MCP server are both ordinary HTTP clients of the same API; the agent reaches it only through the MCP server, inheriting its guards (§12–13).
Three access policies at one edge	Public paths stay anonymous, the MCP path needs a per-client service token, and admin paths need a human login. Co-located hops (agent → MCP server → API) stay on loopback with no extra auth (§16).
Operations are part of the design	The refresh job validates before publishing, restarts the API's caches, and sends exactly one prioritised notification (§15).
Option A and B are alternatives	A project may build either or both of the Streamlit and React front ends; the agent layer is optional and sits on top of the API (§8).

5. Persistence options

Step 3 ("persist the results") has several real options, and picking the right one matters more than it might seem for a small project — it affects how easy the system is to query, scale, back up, and reason about later.

Option	What it is	Pros	Cons	Best fit
Flat files (CSV, JSON, Parquet)	Plain files on disk (or object storage), read/written by whatever data tools you're already using	Simplest possible option; human-readable (CSV/JSON); no server to run; trivial to version-control small ones; Parquet adds columnar compression and typed schema	No concurrent-write safety; no query language (you load the whole file and filter in code); no built-in indexing for large data; CSV has no native types (everything is a string until parsed)	Small-to-medium, mostly-static datasets with a single writer (a batch pipeline) and read-only consumers — the default for the kind of project this series describes
Relational database (PostgreSQL, MySQL, SQLite)	Structured tables with a defined schema, queried with SQL	Strong consistency guarantees; a real query language (filtering, joins, aggregation pushed to the database instead of your application code); mature tooling, indexing, and concurrent access support	Requires running (or paying for) a database server, except SQLite which is serverless but not built for concurrent writers; schema changes need explicit migrations	Data that's queried in varied, ad-hoc ways, updated incrementally, or accessed by multiple writers concurrently
Document / NoSQL store (MongoDB and similar)	Schema-flexible collections of JSON-like documents	No rigid upfront schema — easy to evolve; natural fit for nested/irregular data	Weaker consistency guarantees by default in some systems; query capability is often less expressive than SQL for relational questions (joins across collections)	Data whose shape varies a lot between records, or that's naturally document-shaped (e.g., a user profile with a variable set of fields)
Object storage (S3, GCS, Azure Blob)	Storage for arbitrary files (often the same flat files above), addressed by key, at scale	Effectively unlimited capacity; cheap; the standard place to put large files (raw datasets, model artifacts) in a cloud deployment; often paired with flat files as the actual format	Not a query engine on its own — you still need to download/read a file to look inside it (though some formats/tools support partial reads)	Large files, especially in a cloud-deployed system, or when files need to be shared between services that don't share a filesystem
Data warehouse (BigQuery, Snowflake, Redshift)	A managed, large-scale analytical SQL database, optimized for scanning huge volumes of data for analytics rather than fast single-row lookups	Extremely good at aggregating over large historical datasets; typically fully managed (no server to run yourself)	Cost and complexity that's usually unjustified until data volume is genuinely large; not designed for low-latency single-record lookups	Large-scale analytics with data volumes well beyond what a single machine's memory/disk comfortably handles
In-memory cache (Redis, Memcached)	A fast key-value store, usually used *alongside* one of the above, not instead of it	Very low latency; ideal for caching expensive computed results across multiple server processes	Not durable by default (data can be lost on restart, depending on configuration); not a source of truth on its own	Caching computed API responses across multiple worker processes — see `02-python-api-backend/` for the caching options this project's API layer would use

For the kind of project this series describes — a batch analytical pipeline with read-only downstream consumers and no concurrent writers — **flat files (optionally in a columnar format like Parquet for larger data) are usually the right default**, precisely because they avoid the operational cost of running a database for a problem that doesn't need one. Move to a relational database once you have genuinely concurrent writers, need ad-hoc querying beyond what fits in memory, or need transactional guarantees.

6. Options for the data-processing layer

Option	Pros	Cons
Jupyter notebooks	Best-in-class for interactive, exploratory work — inline visualizations, cell-by-cell iteration, mixing narrative/markdown with code	Poor fit for automated scheduling or CI (execution order can be hidden by out-of-order cell runs); harder to unit-test than plain functions; diffing notebook files in version control is noisy
Plain scripts	Easy to schedule, easy to test (ordinary Python functions/modules), clean version-control diffs	Loses the interactive, visual, iterative feedback loop that makes early exploration fast
Orchestrated pipelines (Airflow, Prefect, Dagster)	Explicit dependency graphs between steps, built-in scheduling/retry/monitoring, a real production-grade answer to "run this pipeline reliably and repeatedly"	Real infrastructure to run and operate; overkill for a one-off or small-scale analysis

A common, sensible progression: **explore in notebooks, extract stable logic into plain functions/scripts once it's no longer changing daily, and only add an orchestrator once the pipeline needs to run reliably and repeatedly without a person watching it.** See the Data Science and ML section for the full data-science curriculum, which is written primarily from the notebook-based, exploratory perspective.

7. Options for the API layer

Option	Pros	Cons
FastAPI (Python)	Type-hint-driven request/response validation; automatic OpenAPI docs; native `async` support; fast to write	Younger ecosystem than Flask/Django (though now very mature and widely adopted)
Flask (Python)	Minimal, unopinionated, huge ecosystem of extensions, very gentle learning curve	No built-in request/response validation or async support out of the box (both are available via extensions, not built in)
Django REST Framework (Python)	Comes with a full web framework (ORM, admin panel, auth) if you need those too	Heavier than necessary if you only need a thin JSON API with no need for Django's other features
Express (Node.js/TypeScript)	JavaScript/TypeScript end-to-end (same language as a React frontend); huge ecosystem	No built-in request/response validation (typically added via a separate library like Zod)

REST vs. GraphQL vs. gRPC (a separate, orthogonal choice from which framework you use): REST (what this series uses throughout) models an API as a set of resource-oriented URLs returning fixed-shape JSON — simple, cacheable, universally understood. GraphQL lets a client request exactly the fields it needs in a single query, which reduces over-fetching for complex, deeply-nested data at the cost of more complex server-side setup.

gRPC uses a compact binary protocol and strongly-typed contracts, favored for high-performance service-to-service communication rather than public/browser-facing APIs. For a read-mostly analytics API serving a small, well-known set of dashboard pages — REST's simplicity usually wins; see the Python API Backend section for the full curriculum built on FastAPI + REST.

8. Options for the presentation layer

Option	Pros	Cons
Single-process dashboard framework (Streamlit, Gradio, Dash)	Fastest path from analysis to interactive UI — one script, no separate server, no frontend build step, direct in-process access to Python objects	Only that one app can use the results; UI is constrained to the framework's built-in widgets; harder to reuse UI pieces across multiple apps
Decoupled SPA framework (React, Vue, Angular)	Full control over UI/UX; a proper component model and ecosystem; the API it depends on can serve other clients too; frontend and backend can be developed, tested, and deployed independently	Real additional cost: an API layer to design and maintain, a separate build pipeline, more moving parts overall
Server-rendered framework (Next.js, Django templates)	Combines some of both — server generates HTML directly (good for SEO, fast initial load) while still supporting rich client-side interactivity in modern frameworks	A different mental model from a pure SPA or a pure dashboard script; adds its own learning curve
Notebook-as-dashboard tools (Voilà and similar)	Turns an existing notebook directly into a shareable app with almost no extra work	Inherits notebook execution-model quirks; least flexible of all the options for genuinely custom UI

This series covers the two most common choices for a project of this shape in detail: a single-process framework (the Streamlit Dashboards section) and a decoupled SPA (the React Front End section). **Neither is strictly better** — pick based on whether the goal is the fastest path to an internal tool, or a production-shaped, independently-evolvable system.

A **conversational agent** (§13) is a third, optional presentation surface, layered on top of whichever of the two above already exists rather than replacing it — typically surfaced as one more view inside the same client application (§8's decoupled SPA option is the natural host for it), answering open-ended natural-language questions through an MCP server and agent rather than through fixed charts and controls. See §12–13.

9. Why the persistence layer and the presentation layer(s) stay separate

Whichever presentation approach is used, the data science layer's job ends at "produce a durable, well-defined set of outputs" — it should not need to know anything about how those outputs get displayed. This separation matters for concrete reasons:

- **Reproducibility.** Analysis and model training can be expensive (minutes to hours). If the presentation layer had to re-run the analysis on every page load, every interaction would be slow and non-deterministic. Persisting outputs once and re-reading them is far cheaper and keeps results stable between runs.

- **Multiple consumers.** A well-defined output contract can be read by more than one thing — a quick internal dashboard *and* a production client — without duplicating the analysis logic.
- **Independent iteration.** A data scientist can change how a model is trained without touching the frontend, as long as the *output contract* stays the same; a frontend developer can redesign a chart without needing to understand the modeling code at all.
- **Graceful degradation.** If a downstream stage hasn't run yet, the presentation layer should be able to say so clearly (a "not ready yet" message or an HTTP 503) rather than crash — this only works cleanly if "the data isn't there" is a condition the presentation layer can check for, which requires genuine decoupling.

10. A typical constants/contract discipline

Whatever the underlying data represents (which entities are in scope, what the target variable is, what date range matters), that set of decisions tends to get defined **once**, early, and referenced everywhere downstream. When a project spans multiple languages/runtimes (a Python data-processing layer, a Python API, a TypeScript frontend), that single source of truth usually has to be **hand-mirrored** once per language (§2.3) — plan for that as a real, tracked maintenance surface.

Similarly, whatever the API layer returns as its response shape becomes a contract the frontend depends on. Changing a field name or removing a field on one side without updating the other is one of the most common integration bugs in a decoupled architecture — worth checking for explicitly any time a response shape changes.

11. Deployment topology — how these layers typically get deployed together

Each of the technology-specific documents below has its own **Deployment** module covering the details; at a high level, a system built this way typically has:

- **A build/CI step** for each independently-deployed piece (the API, and separately the frontend build, if using the decoupled option) — run tests, then produce a deployable artifact (a container image, a static asset bundle).
- **A reverse proxy or gateway** in front of the deployed services, which is usually where concerns like HTTPS termination, routing multiple services under one public domain (e.g., serving the frontend at `/` and proxying `/api/*` to the backend), and any URL-prefix rewriting live.
- **Environment-specific configuration** (§2.7) injected at deploy time — a database connection string, an API base URL, feature flags — never baked into the build artifact itself.
- **Separate deployability for each layer**, even if they're deployed to the same infrastructure — being able to redeploy the frontend without redeploying the API (and vice versa) is one of the concrete payoffs of the decoupled architecture (§8).

12. Options for exposing an API to an LLM agent (MCP)

Once step 4's API exists, there's a further, optional choice about whether — and how — to make its results callable by an LLM, not just fetchable by a frontend.

Option	What it is	Trade-off
A dedicated MCP server (this series' choice — see `05-mcp-server/`)	A standard-protocol server wrapping the API as tools, reusable by any MCP-compliant host (a desktop LLM app, a custom agent)	The most setup, but solves the M×N integration problem once, for every future consumer — see `05-mcp-server/01-core-concepts.md`
Auto-generated tools from an OpenAPI spec	Some tooling can convert an existing OpenAPI/REST spec directly into tool definitions with no hand-authoring	Fast to stand up, but a 1:1 conversion rarely produces tools shaped the way a model actually asks questions — see `05-mcp-server/02-tool-design.md` on why hand-curated, sometimes composed tools usually answer real questions better
Bespoke, app-specific tool-calling code	Skip a standard protocol entirely and wire tool-calling directly into one specific agent's own code	Fastest for a single, one-off agent; loses MCP's whole reusability benefit the moment a second consumer (a different host, a second agent) shows up

For any project expecting more than one LLM consumer of the same underlying data — even just "manual verification via a desktop app, then later a custom agent," which is a common progression — a dedicated MCP server is worth the extra setup specifically because it only has to be built once. See the MCP Servers section for the full curriculum.

13. Options for the conversational-agent layer

Option	What it is	Trade-off
A graph-based orchestration framework (LangGraph and similar — this series' choice, see `06-conversational-agent/`)	Explicit nodes/edges/conditional routing, a typed shared state, checkpointer-backed memory	The most structure for a genuinely multi-step agent (classification, a bounded tool-calling loop, a separate response-composition step) — see `06-conversational-agent/01-core-concepts.md` for why a graph earns its keep over a plain loop
A single-agent SDK loop (a provider's own agent-loop helper, with no explicit graph)	The model + bound tools + an implicit "keep calling until done" loop, with less orchestration code to write	Simpler for a genuinely single-path agent; branching logic (guardrail classification routing to entirely different handling) tends to end up as nested conditionals instead of explicit graph structure once the agent grows past the simplest case
A hand-rolled loop	A plain `while` loop around "call the model, execute any requested tool, repeat"	The least dependency overhead; reasonable for a genuinely minimal prototype, but reimplements (often incompletely) state management, memory, and streaming that a framework already provides

Whichever is chosen, the concerns in `06-conversational-agent/02-tool-calling-and-guardrails.md` — classifying requests before acting, bounding the tool-calling loop, and mapping results to real UI components rather than freeform generated markup — apply regardless of the specific orchestration mechanism underneath.

14. Choosing where logic lives: a rule of thumb

As a system grows, the recurring question is *which layer should compute this?* A workable rule: compute **once**, in the **earliest** layer that has the information, and let every later layer read it.

Kind of logic	Lives in	Why
Statistics, fits, derived series, scenario translations	The pipeline (data layer)	Expensive, must be reproducible, and must not differ between consumers
Validation of external data, licence and provenance records	The pipeline	It is the only layer that sees the raw source
Serving, filtering by parameter, fail-closed errors, envelope	The API	Stateless reads over what the pipeline wrote
Formatting numbers, layout, URL state, accessibility	The frontend	Presentation concerns; the API sends raw values with units
Tool shaping, argument resolution, summaries for a model	The MCP server	The model-facing contract, consumer-agnostic
Required disclosures, answer blocks, follow-up links	The agent	Per-answer composition, but computed in code (§2.10)
Rate limits, access policy	The edge	Enforced before a request reaches the host

When the same fact appears in two layers (a caveat in the pipeline *and* retyped in a tooltip), it will eventually disagree — pick the earliest layer and make the later one read it.

15. Operating the whole system

A running product is more than its code, and a few operational habits recur across every layer:

- **A scheduled refresh owns freshness.** One job downloads the sources, validates each against the last good copy (hard-fail → restore the backup; soft-flag → publish and alert), re-runs the notebook/pipeline stages with each stage a separate failure domain (a stage that reads another declares the dependency and is *skipped as a failure* if that stage failed), restarts the processes that cache in memory (the API's loaders), and sends **one notification per run** whose priority is the highest of everything that happened.
- **Deploy order follows the dependency graph.** A frontend that consumes a shared design system from source needs that checkout updated before it is built; a service whose environment changed needs its service-manager job reloaded, not merely restarted.
- **Know when the build is the release.** If production serves the build directory directly, running the build is the point of no return (React Deployment, §8).
- **Verify with evidence from outside.** The content-hashed asset filename the live page loads; a 403 for a wrong token from a machine that is not the host; a service worker unregistered before judging a frontend release.
- **Keep a rollback path for each deploy**, and write it down with the deploy record.
- **Document in two layers.** A current-state architecture document updated only on architecturally-significant changes, and a separate chronological changelog for history. Write the design for a change *before*

implementing it; update it afterward with what was actually found.

16. Access control and admin at the edge

Most of a public analytics product is deliberately unauthenticated — the data is meant to be public, and a secret shipped to a browser protects nothing. That doesn't mean "no access control"; it means deciding it **per trust boundary**:

Boundary	Typical decision
Browser → public app and API	Anonymous. Protect with edge rate limits and security headers, not a token
Service → the API it wraps, same host	Network isolation (bind to loopback); a token with nothing to validate it is dead code
External machine clients → an MCP server	Edge service-auth policy with one named, revocable token per client (MCP Deployment, §2.1)
A co-located agent → the MCP server	The loopback case, not the external-client case
A human operator → an admin page	Edge **login** policy through an identity provider, one allow-listed account

Admin capabilities follow one rule: **each service owns its own admin routes** (the service that owns the setting), and a single edge application gates them all by path — adding a future capability means adding a path rule, not building new auth infrastructure. Gate the *page* as well as the API (a login policy redirects, and only a real navigation can follow a redirect), keep the page out of navigation and out of the service worker's fallback, and see Agent Deployment, §8 for a safe runtime-switch design. Don't expose new surface (interactive docs, schema routes, admin paths) on a public, unauthenticated API through the public route until it has auth.

17. What to read next

- Building the data engineering / modeling / forecasting pipeline → `01-data-science-ml/` (and, for several external sources kept current on a schedule, `05-multi-source-pipelines.md`)
- Building a backend API to serve results → `02-python-api-backend/`
- Building a quick, single-process dashboard → `03-streamlit-frontend/`
- Building a decoupled, component-based frontend → `04-react-frontend/` (and `05-data-heavy-dashboard-patterns.md` once it has many pages and a phone audience)
- Exposing results to an LLM as tools → `05-mcp-server/`
- Building a conversational agent on top of those tools → `06-conversational-agent/`

The training series README lists every area's documents in reading order; each area assumes you've read this document first.